Lesson 1: Intro to the ESP32

TABLE OF CONTENTS

1. Programming environment
2. Comparison Table
3. The Adafruit ESP32 Series
    a. The Adafruit ESP32 Specs
    b. ESP32 pin list
    c. Huzzah32 pin diagram
    d. Important notes
    e. ADC2 is only usable when WiFi not activated
    f. Huzzah32 installation instructions for the Arduino IDE
        a. Step 1: Add ESP32 to Arduino Board Manager
        b. Step 2: Install USB to UART Bridge Virtual COM Port Driver
        c. Step 3: Select “Adafruit ESP32 Feather” in board menu
        d. Step 4: Select the appropriate port
4. Resources
    a. Official ESP32 Documentation
    b. Other
5. Next Lesson



Image from makeradvisor.com. There are literally dozens of ESP32 boards. Search online for comparisons (*e.g.*, link).

The ESP32 is a low-cost, "system-on-a-chip" board with integrated WiFi, Bluetooth, ultra-low power consumption, and a fast microprocessor created by Espressif. The ESP32 is the successor to Espressif's massively successful ESP8266 but is far more powerful and feature rich.

The original ESP32 was released in 2016 but since then there have been dozens of variants and updates created by Espressif. The latest board is the ESP32-S3 released in 2020, which features a dual-core Xtensa® 32-bit LX7 running up to 240MHz, has built-in WiFi and Bluetooth, up to 8MB flash memory, and lots of peripheral support. In addition, there are now micro-variants like the ESP32-C series, which have smaller footprints. See Espressif's product comparison tool.

Programming environment

You can program the ESP32 in variety of languages and programming environments, including C/C++, Micropython, Lua, and more. The ESP32 is completely **independent** of the Arduino ecosystem. Just as you don't have to use Arduino to program the ATmega328P (used in the Arduino Uno) or the ATmega32U4 (found in the Arduino Leonardo), you don't have to use Arduino to program the ESP32.

For our learning series, however, we *will* be using Arduino to program the ESP32—and so most of our prior learning will directly transfer (woohoo! 🎉). We can use Arduino to program the ESP32 because Espressif has created an open-source Arduino core for the ESP32 series. See Espressif's official "Getting Started" guide. Note: not all of ESP32's functionality is exposed via this library, you can see limitations here.

If you want to try and program your ESP32 *without* Arduino, follow the Espressif ESP-IDF Programming Guide . We have never done this, so this approach is beyond the scope of our learning series.

Comparison Table

Here's a comparison table of the Arduino Uno Rev3 and ESP32s derived from Espressif's official documentation and this comparison table on GitHub Gist.

> **NOTE**
> While there are many differences between the Arduino Uno/Leonardo and the ESP32, a major one is that the ESP32s run at 3.3V rather than 5V. This has implications for how you interface with electronic components via the GPIO pins.

Feature	Arduino Uno	ESP32	ESP32-S2	ESP32-S3
Launch year	2010	2016	2020	2020
Datasheet	Uno Datasheet (PDF)	ESP Datasheet (PDF)	ESP-S2 Datasheet (PDF)	ESP-S3 Datasheet (PDF)
Core	ATmega328P	Xtensa® dual-/single core 32-bit LX6	Xtensa® single-core 32-bit LX7	Xtensa® dual-core 32-bit LX7
Wi-Fi protocols	✕	802.11 b/g/n, 2.4 GHz	802.11 b/g/n, 2.4 GHz	802.11 b/g/n, 2.4 GHz
Bluetooth®	✕	Bluetooth v4.2 & BLE	✕	Bluetooth 5.0
Typical frequency	16 MHz	240 MHz	240 MHz	240 MHz
SRAM	2 KB	520 KB	320 KB	512 KB
ROM	32 KB	448 KB	128 KB	384 KB
Embedded flash	32 KB	2 MB, 4 MB, or none	2 MB, 4 MB, or none	8 MB or none
External flash	✕	Up to 16 MB device	Up to 1 GB device	Up to 1 GB device
External RAM	✕	Up to 8 MB device	Up to 1 GB device	Up to 1 GB device

And peripheral support. Note that the ADCs on the ESP32s are 12-bit rather than 10-bit so they ostensibly provide far more analog-to-digital precision: 3.3V is linearly divided across 0-4095 (10 bits).

Peripherals	Arduino Uno	ESP32	ESP32-S2	ESP32-S3
ADC	One 10-bit, 6 channels	Two 12-bit, 18 channels	Two 12-bit, 20 channels	Two 12-bit SAR ADCs, 20 channels
DAC	✕	Two 8-bit channels	Two 8-bit channels	✕
Timers	Three 16-bit	Four 64-bit	Four 64-bit	Four 54-bit
Watchdog Timers	1	Three	Three	Three
Temperature sensor	✕	✕	1	1
Touch sensor	✕	10	14	14
Hall sensor	✕	1	✕	✕
GPIO	14	34	43	45
SPI	1	4	4	4

Peripherals	Arduino Uno	ESP32	ESP32-S2	ESP32-S3
LCD interface	✕	1	1	1
UART	1	3	2	3
I2C	1	2	2	2
I2S	✕	2, can be configured to operate with 8/16/32/40/48-bit resolution as an input or output channel.	1, can be configured to operate with 8/16/24/32/48/64-bit resolution as an input or output channel.	2, can be configured to operate with 8/16/24/32-bit resolution as an input or output channel.
Camera interface	✕	1	1	1
Pulse counter	✕	8 channels	4 channels	4 channels
LED PWM	✕	16 channels	8 channels	8 channels
Motor Control PWM	✕	2, six PWM outputs	✕	2, six PWM outputs

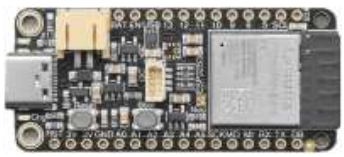
The Adafruit ESP32 Series

For our tutorial series, we will be using **Adafruit's ESP32 boards**. Specifically, Adafruit's original ESP32 board called the Huzzah32 ESP32 Feather, which came out in May 2017, as well as Adafruit's ESP32-S3 board released in Nov, 2021.

While you can find (far) cheaper ESP32 alternatives on AliExpress or Amazon—down to only a few dollars—Adafruit produces reliable, high-quality products and has good customer support (check out the Adafruit forums). In addition, just as the Arduino Uno's standardized form factor created an eco-system of stackable expansion boards so too has **Adafruit created stackable shields** for "Feather"-compatible boards like the ESP32. Adafruit calls these expansion boards Wings—see a listing here and an overview of the FeatherWings here). For example, you can add an MP3 Player FeatherWing, a GPS FeatherWing, or a DC Motor FeatherWing, to name a few.

Regardless of which ESP32 board you use, our tutorial series should help. Note, however, that pin locations may be different.

The Adafruit ESP32 Specs

Name	Arduino Uno	Adafruit ESP32	Adafruit ESP32-S3
Image			

Name	Arduino Uno	Adafruit ESP32	Adafruit ESP32-S3
Store	Arduino	Adafruit	Adafruit
Microcontroller	8-bit, 16 MHz ATmega328P	32-bit, 240 MHz dual core Tensilica LX6	32-bit, 240 MHz dual core Tensilica LX7
Input voltage (limit)	6-20V	Use either USB (5V) or 3.7V LiPoly	5V (via USB) or 3.7V (LiPoly)
Operating voltage	5V	3.3V	3.3V
Flash memory	32KB	4MB	Up to 8MB
SRAM	2KB	520KB	Up to 2MB
GPIO pins	14	21	25
PWM pins	6	All	All
Analog inputs	6	14	20
Wi-Fi	N/A	802.11b/g/n HT40 Wi-Fi transceiver	802.11b/g/n HT40 Wi-Fi transceiver
Bluetooth	N/A	Dual mode (classic and BLE)	Bluetooth 5.0 (LE)
USB Connection	Type-B (rectangular connection used on printers)	MicroUSB	USB-C

Unlike the ATmega chips used in the Uno and Leonardo, there is a hardware floating point unit (FPU) on the ESP32; however, there have been some criticisms about its performance (link1, link2). Our understanding is that FPU performance has been improved on the ESP32-S3s (and supports both single and double precision).

The Huzzah32 is **not** designed for external power supplies, so either use the USB port with a 5V 1A USB wall adapter or by plugging into your computer or a LiPoly battery (3.7/4.2V). Unlike with the Arduino Uno and Leonardo, do not use a 9V battery or you could damage your board!

ESP32 pin list

The official ESP32 pin list is here:

PIN Name	Power	Analog Function1	Analog Function2	Analog Function3	RTC_GPIO	Type	Function1	Type	Function2	Type	Function3	Type	Function4	Type	Function5	Type	At Reset	After Reset
SENSOR_VP	VRTC	ADC_H	ADC1_CH0		RTC_GPIO0	I	GPIO36	I			GPIO36	I					oe=0	
SENSOR_CAPP	VRTC	ADC_H	ADC1_CH1		RTC_GPIO1	I	GPIO37	I			GPIO37	I					oe=0	
SENSOR_CAPN	VRTC	ADC_H	ADC1_CH2		RTC_GPIO2	I	GPIO38	I			GPIO38	I					oe=0	
SENSOR_VN	VRTC	ADC_H	ADC1_CH3		RTC_GPIO3	I	GPIO39	I			GPIO39	I					oe=0	
VDET_1	VRTC		ADC1_CH6		RTC_GPIO4	I	GPIO34	I			GPIO34	I					oe=0	
VDET_2	VRTC		ADC1_CH7		RTC_GPIO5	I	GPIO35	I			GPIO35	I					oe=0	
32K_XP	VRTC	XTAL_32K_P	ADC1_CH4	TOUCH9	RTC_GPIO9	I/O/T	GPIO32	I/O/T			GPIO32	I/O/T					oe=0	
32K_XN	VRTC	XTAL_32K_N	ADC1_CH5	TOUCH8	RTC_GPIO8	I/O/T	GPIO33	I/O/T			GPIO33	I/O/T					oe=0	
GPIO25	VRTC	DAC_1	ADC2_CH8		RTC_GPIO6	I/O/T	GPIO25	I/O/T			GPIO25	I/O/T					oe=0	
GPIO26	VRTC	DAC_2	ADC2_CH9		RTC_GPIO7	I/O/T	GPIO26	I/O/T			GPIO26	I/O/T					oe=0	
GPIO27	VRTC		ADC2_CH7	TOUCH7	RTC_GPIO17	I/O/T	GPIO27	I/O/T			GPIO27	I/O/T					oe=0	
MTMS	VRTC		ADC2_CH6	TOUCH6	RTC_GPIO16	I/O/T	MTMS	I0	HSPICLK	I/O/T	GPIO14	I/O/T					oe=0, wpu, ie=1	wpu, ie=1
MTDI	VRTC		ADC2_CH5	TOUCH5	RTC_GPIO15	I/O/T	MTDI	I1	HSPIQ	I/O/T	GPIO12	I/O/T					oe=0, wpd, ie=1	wpd, ie=1
MTCK	VRTC		ADC2_CH4	TOUCH4	RTC_GPIO14	I/O/T	MTCK	I1	HSPID	I/O/T	GPIO13	I/O/T	U0CTS	I1			oe=0, wpd, ie=1	wpd, ie=1
MTDO	VRTC		ADC2_CH3	TOUCH3	RTC_GPIO13	I/O/T	MTDO	O/T	HSPICS0	I/O/T	GPIO15	I/O/T	U0RTS	O			oe=0, wpu, ie=1	wpu, ie=1
GPIO2	VRTC		ADC2_CH2	TOUCH2	RTC_GPIO12	I/O/T	GPIO2	I/O/T	HSPIWP	I/O/T	GPIO2	I/O/T					oe=0, wpd, ie=1	wpd, ie=1
GPIO0	VRTC		ADC2_CH1	TOUCH1	RTC_GPIO11	I/O/T	GPIO0	I/O/T	CLK_OUT1	O	GPIO0	I/O/T					oe=0, wpu, ie=1	wpu, ie=1
GPIO4	VRTC		ADC2_CH0	TOUCH0	RTC_GPIO10	I/O/T	GPIO4	I/O/T	HSPIHD	I/O/T	GPIO4	I/O/T					oe=0, wpd, ie=1	wpd, ie=1
GPIO16	VSDIO						GPIO16	I/O/T			GPIO16	I/O/T	HS1_DATA4	I/O/T			oe=0	
GPIO17	VSDIO						GPIO17	I/O/T			GPIO17	I/O/T	HS1_DATA5	I/O/T			oe=0	
SD_DATA_2	VSDIO						SD_DATA2	I1/O/T	SPIHD	I/O/T	GPIO9	I/O/T	HS1_DATA2	I/O/T	U1RXD	I1	oe=0, wpu, ie=1	wpu, ie=1
SD_DATA_3	VSDIO						SD_DATA3	I0/O/T	SPIWP	I/O/T	GPIO10	I/O/T	HS1_DATA3	I/O/T	U1TXD	O	oe=0, wpu, ie=1	wpu, ie=1
SD_CMD	VSDIO						SD_CMD	I1/O/T	SPICS0	I/O/T	GPIO11	I/O/T	HS1_CMD	I/O/T	U1RTS	O	oe=0, wpu, ie=1	wpu, ie=1
SD_CLK	VSDIO						SD_CLK	I0	SPICLK	I/O/T	GPIO6	I/O/T	HS1_CLK	O	U1CTS	I1	oe=0, wpu, ie=1	wpu, ie=1
SD_DATA_0	VSDIO						SD_DATA0	I1/O/T	SPIQ	I/O/T	GPIO7	I/O/T	HS1_DATA0	I/O/T			oe=0, wpu, ie=1	wpu, ie=1
SD_DATA_1	VSDIO						SD_DATA1	I1/O/T	SPID	I/O/T	GPIO8	I/O/T	HS1_DATA1	I/O/T			oe=0, wpu, ie=1	wpu, ie=1
GPIO5	VSDIO						GPIO5	I/O/T	VSPICS0	I/O/T	GPIO5	I/O/T	HS1_DATA6	I/O/T			oe=0, wpu, ie=1	wpu, ie=1
GPIO18	VSDIO						GPIO18	I/O/T	VSPICLK	I/O/T	GPIO18	I/O/T	HS1_DATA7	I/O/T			oe=0	
GPIO23	VSDIO						GPIO23	I/O/T			GPIO23	I/O/T					oe=0	
GPIO19	VIO						GPIO19	I/O/T	VSPIQ	I/O/T	GPIO19	I/O/T	HS2_DATA2	I/O/T			oe=0	
GPIO20	VIO						GPIO20	I/O/T	VSPID	I/O/T	GPIO20	I/O/T	HS2_DATA3	I/O/T			oe=0	
GPIO21	VIO						GPIO21	I/O/T	VSPIHD	I/O/T	GPIO21	I/O/T	HS2_CMD	I/O/T			oe=0	
GPIO22	VIO						GPIO22	I/O/T	VSPIWP	I/O/T	GPIO22	I/O/T	HS2_CLK	O			oe=0	
U0RXD	VIO						U0RXD	I1	CLK_OUT2	O	GPIO3	I/O/T	HS2_DATA0	I/O/T			oe=0, wpu, ie=1	wpu, ie=1
U0TXD	VIO						U0TXD	O	CLK_OUT3	O	GPIO1	I/O/T	HS2_DATA1	I/O/T			oe=0, wpu, ie=1	wpu, ie=1

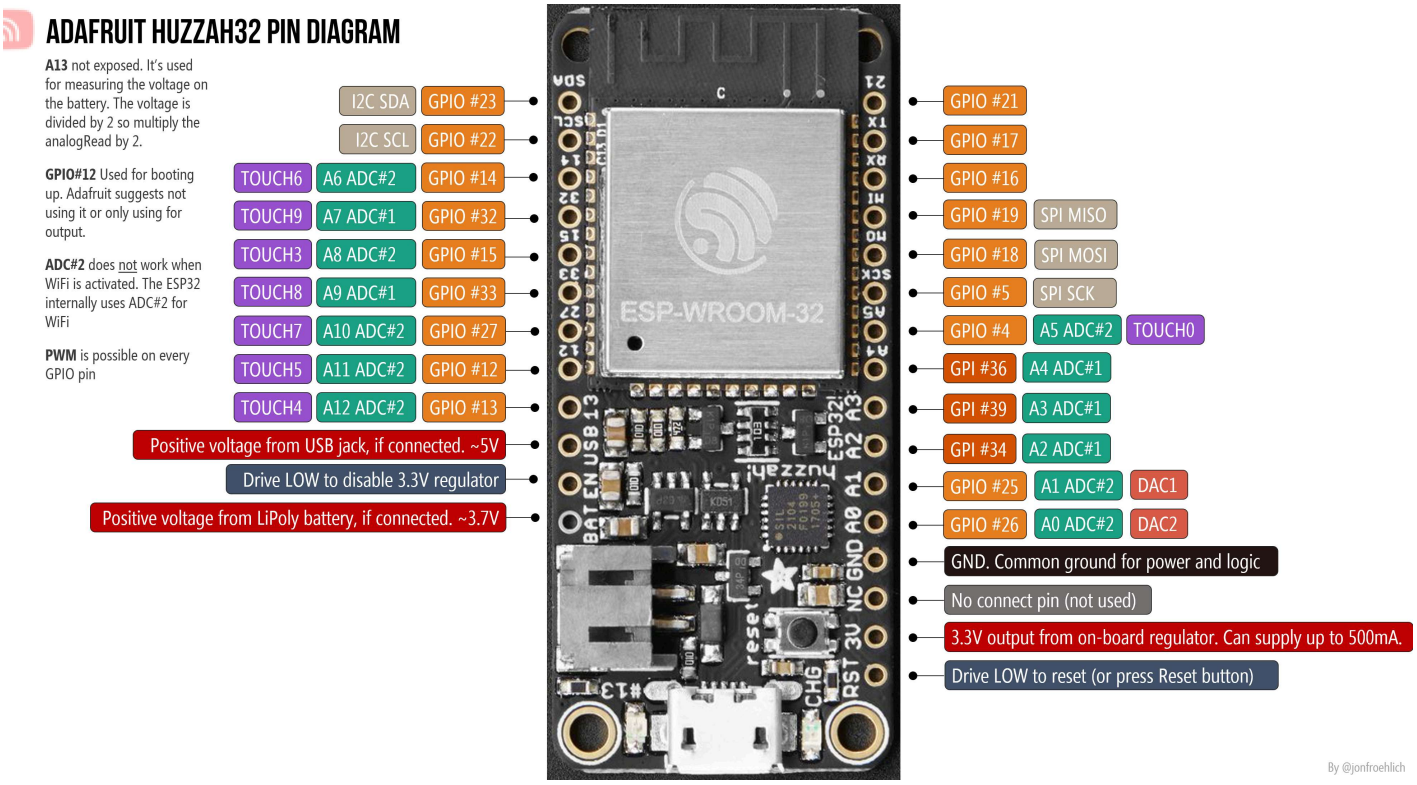
Screenshot of the ESP32 pin list PDF.

In our code, we will reference the pins based on their GPIO number, their analog input number (prefixed by 'A') for analog input, or their touch number (prefixed by 'T') for using capacitive touch sensing. We can always use the GPIO number, however (which is just an integer). The ESP32 datasheet often uses the pin name (far left column of the above pin list) to refer to pins.

Huzzah32 pin diagram

So, what do all of these pins do? Oh, so many things!

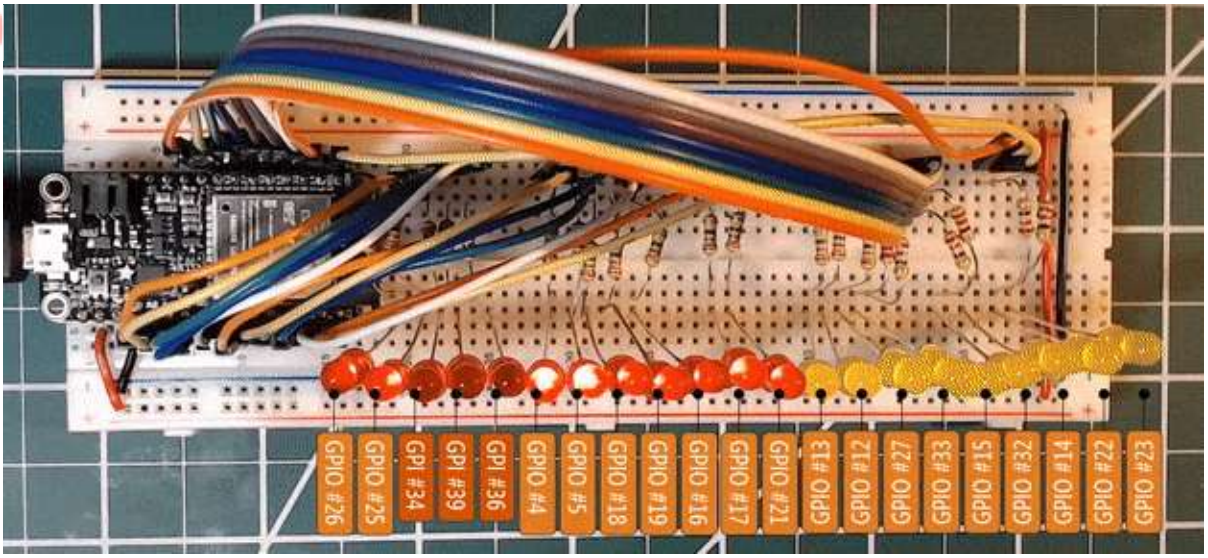
The pin diagram for the Huzzah32 in the official Adafruit docs is pretty confusing. So, we created our own:



See the Adafruit Huzzah32 docs for details. Right-click and open image in a new tab to zoom in.

Important notes

- The ESP32 runs on **3.3V power and logic**, and unless otherwise specified, GPIO pins are not 5V safe! The **BAT** pin should generally not be used directly as the Huzzah32 has a JST connection for the LiPoly battery. You should most definitely not hook up a 9V connection here.
- There are **21 GPIO pins**; however, on the Huzzah32, pins 34 (A2), 39 (A3), 36 (A4) are not output-capable and thus should only be used for input. So, **18 GPIO pins** in total. Be forewarned: the pins are in a strange order, so read the diagram carefully.
- **PWM** is possible on all 18 GPIO pins
- **14 of the 21 GPIO pins** can be used **analog input pins**; however, A13 is not exposed. It's used for measuring the voltage on the LiPoly battery via a voltage divider. When reading in the battery level using `analogRead(A13)`, make sure to multiply by 2 to get correct reading. Here's an initial program to read and print out the battery level to Serial (link)
- The **ADC resolution is 12 bits** (0-4095). This is in contrast to the Arduino Uno and Leonardo, which uses ATmega chips with 10 bit ADCs (so, 0-1023). Make sure you use the proper max value in your conversions (*e.g.*, using `map()`)
- **GPIO 13** is the `LED_BUILTIN` (the red LED next to micro USB)
- The charging circuit light will flash quickly when there is no LiPoly battery plugged in. It's harmless and doesn't mean anything. This LED will also flash (more slowly) when the battery is plugged in and charging. The battery charges automatically when plugged in and the Huzzah32 is externally powered.
- Only power your Huzzah32 either using the USB plug (max 5V, 1A) or a LiPoly battery (3.7/4.2V)



The Huzzah32 has 21 GPIO pins; however pins 34 (A2), 39 (A3), 36 (A4) are not output-capable. In this animation, we are attempting to fade in/out all 21 GPIO pins and demonstrating that only 18 work for output.

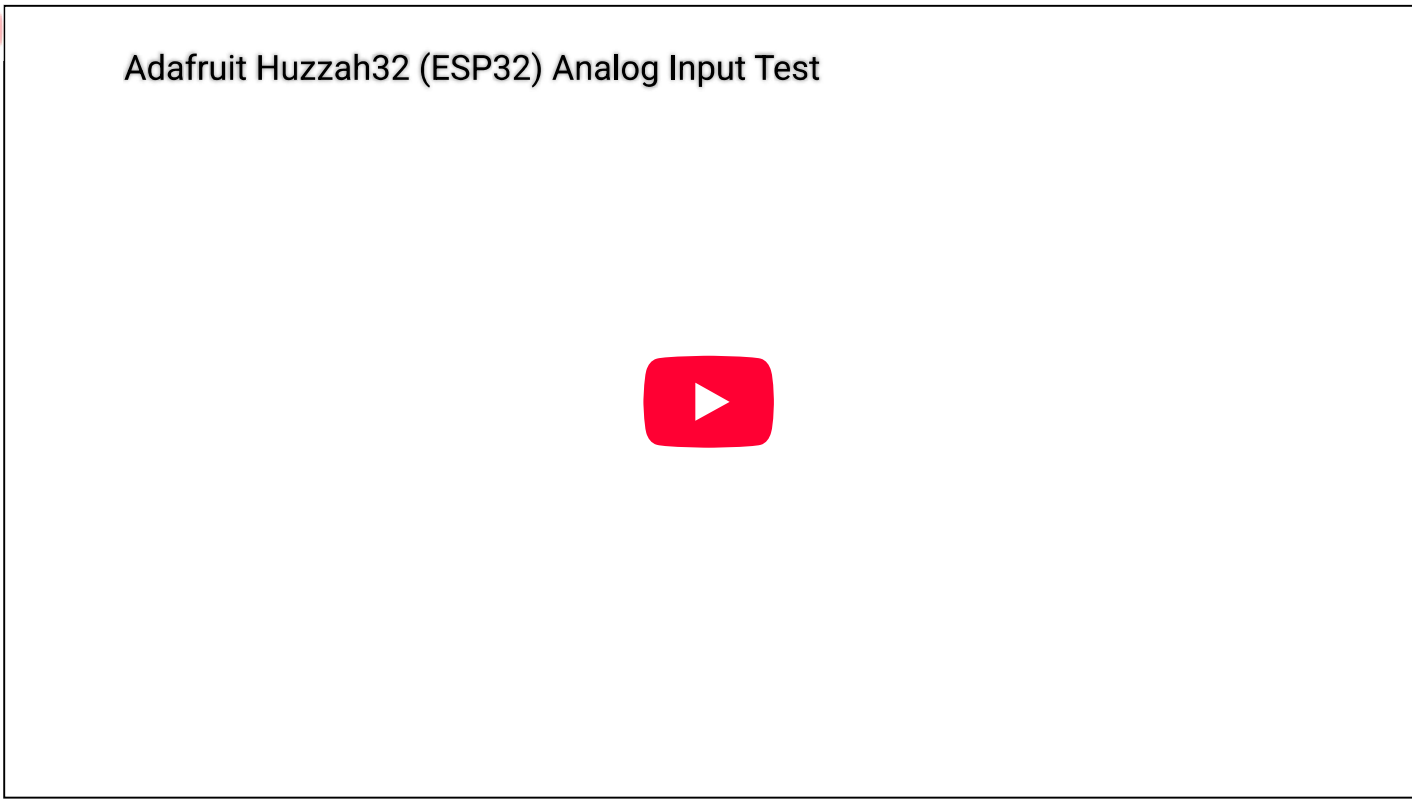
ADC2 is only usable when WiFi not activated

The Adafruit docs state (somewhat confusingly) that ADC#1 only works when WiFi has started. We empirically tested this (see video below) and found this **not** to be true. On the other hand, the Espressif docs state that ADC#2 only works when WiFi has **not** started. We **did** find this to be true. So, we believe the Espressif docs are right and the Adafruit docs are wrong. **Update:** it turns out that the Adafruit docs *are* accurate but just poorly phrased. Indeed, ADC#1 is the only ADC that works when using WiFi (see Reddit post).

Indeed, we empirically tested this. Check out our two programs:

- AnalogInputTest.ino reads from all analog input pins and prints the values to Serial (so you can view them in Serial Console or Serial Plotter)
- WiFiAnalogInputTest.ino extends AnalogInputTest but turns on WiFi.

In the following video, I'm using AnalogInputTest.ino to test all 13 analog input pins (`A0` - `A12`) using a trim potentiometer for input and the Serial Plotter for output.



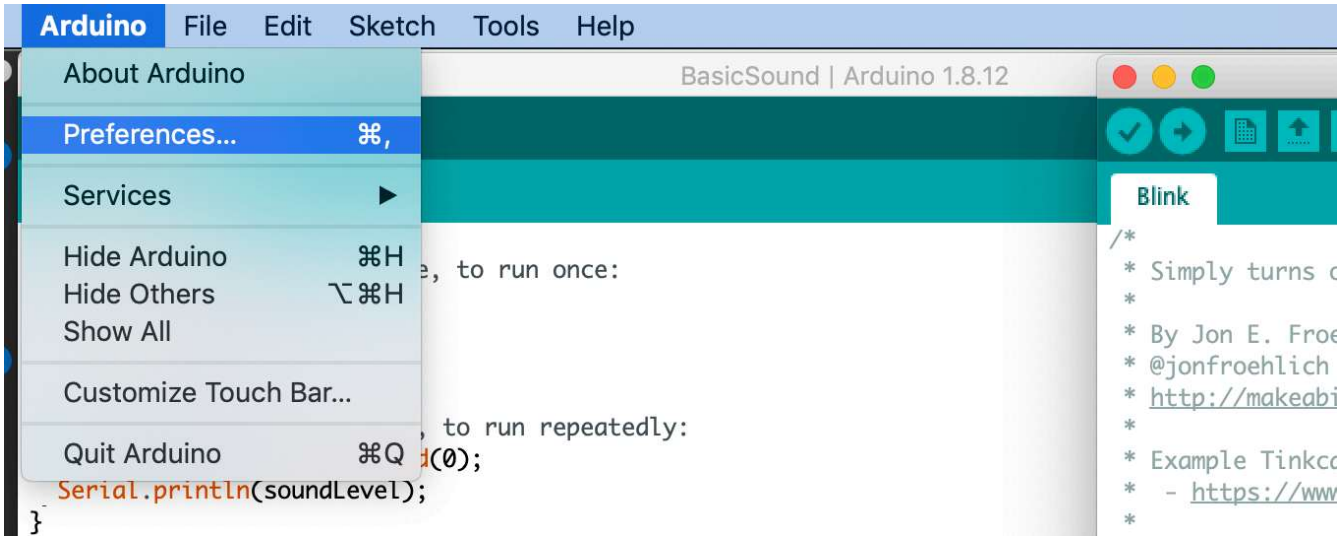
Huzzah32 installation instructions for the Arduino IDE

You can follow the official Adafruit Huzzah32 Arduino IDE installation instructions, which we've expanded a bit below.

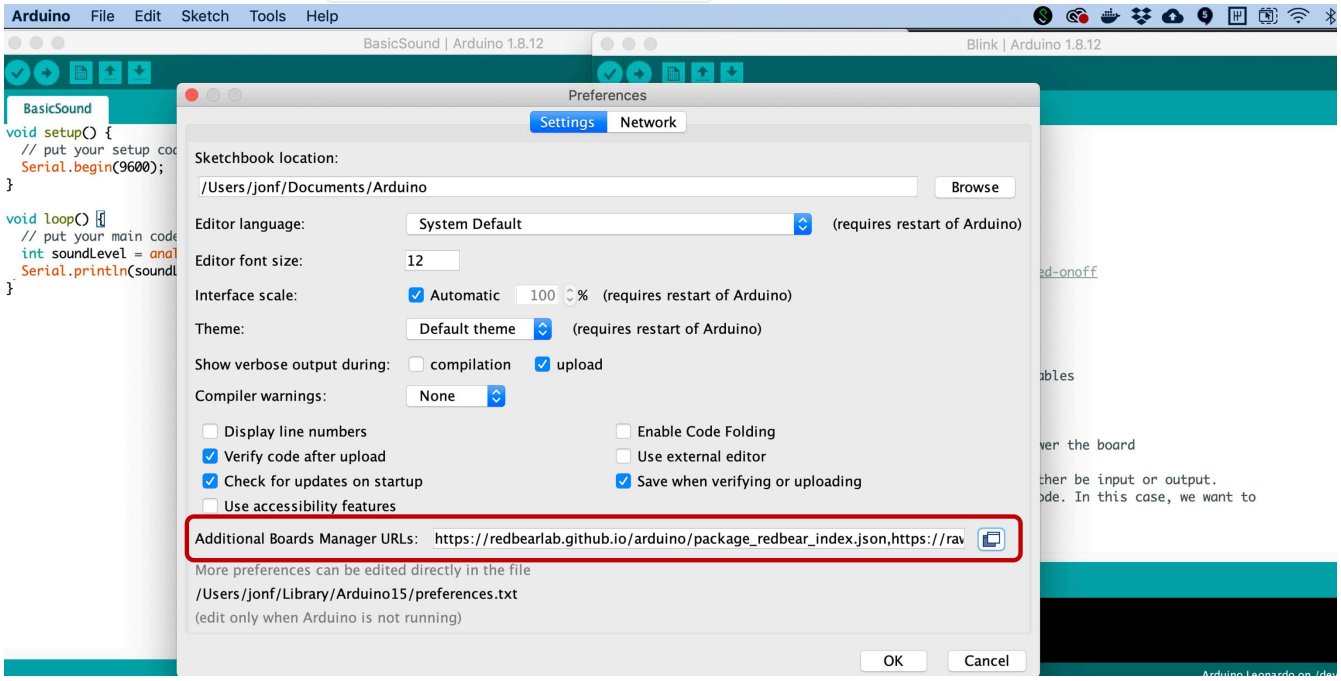
STEP 1: ADD ESP32 TO ARDUINO BOARD MANAGER

1 Open the Arduino IDE

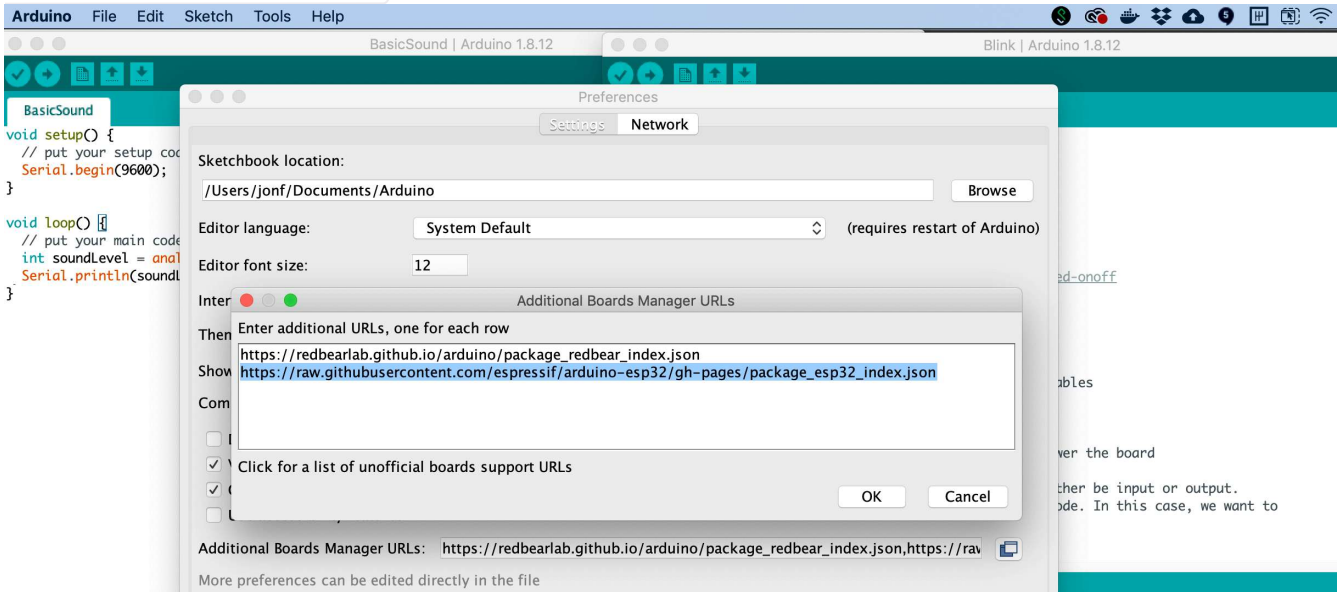
2 Go to Preferences



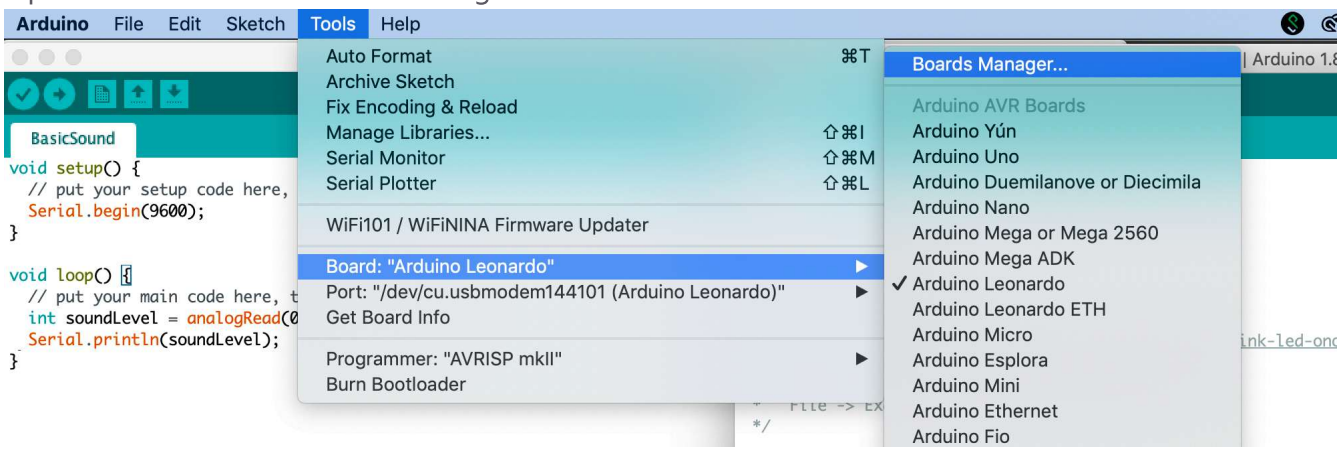
3 In preferences, find the `Additional Board Manager URLs:` field



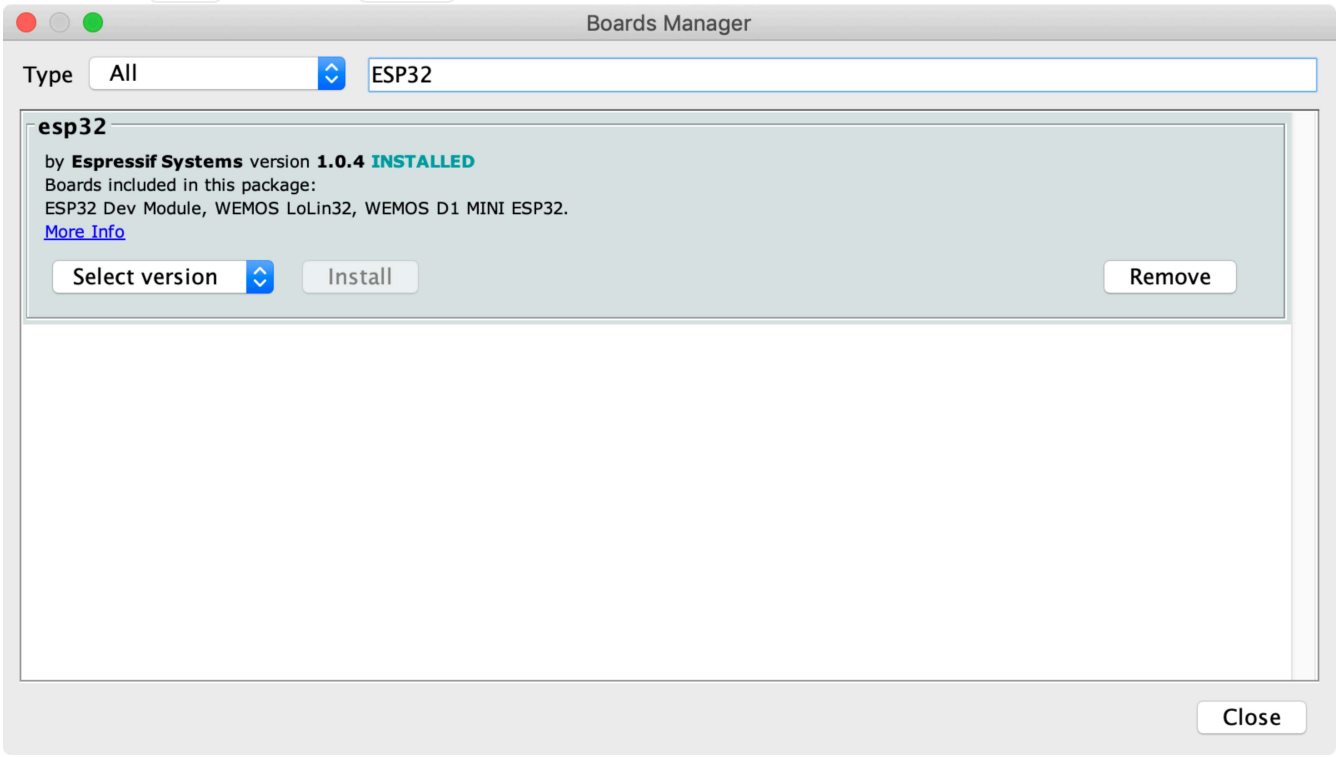
4 Add the ESP32 JSON url `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`



5 Open the Arduino IDE Board Manager



6 Search for `ESP32` and click `Install`

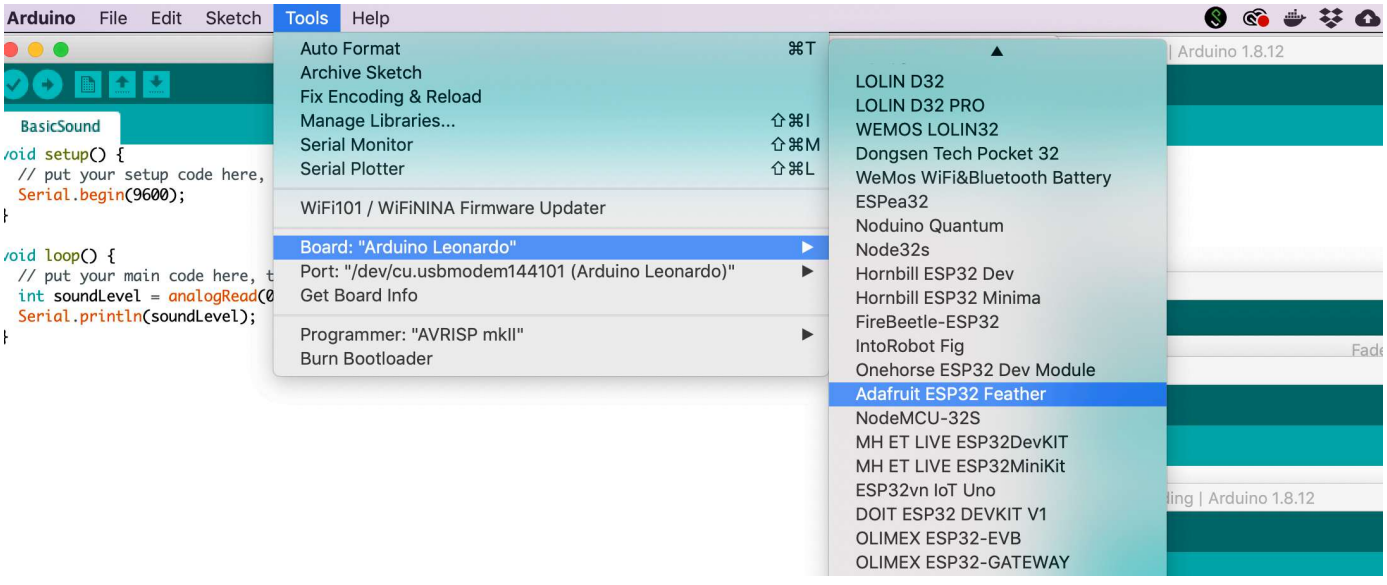


STEP 2: INSTALL USB TO UART BRIDGE VIRTUAL COM PORT DRIVER

As noted in the official Adafruit Huzzah32 Arduino IDE installation instructions, the second step is to install the USB to UART Bridge Virtual COM Port (VCP) driver to interface with the ESP32 board. You can download the driver from Windows, Mac, and Linux here.

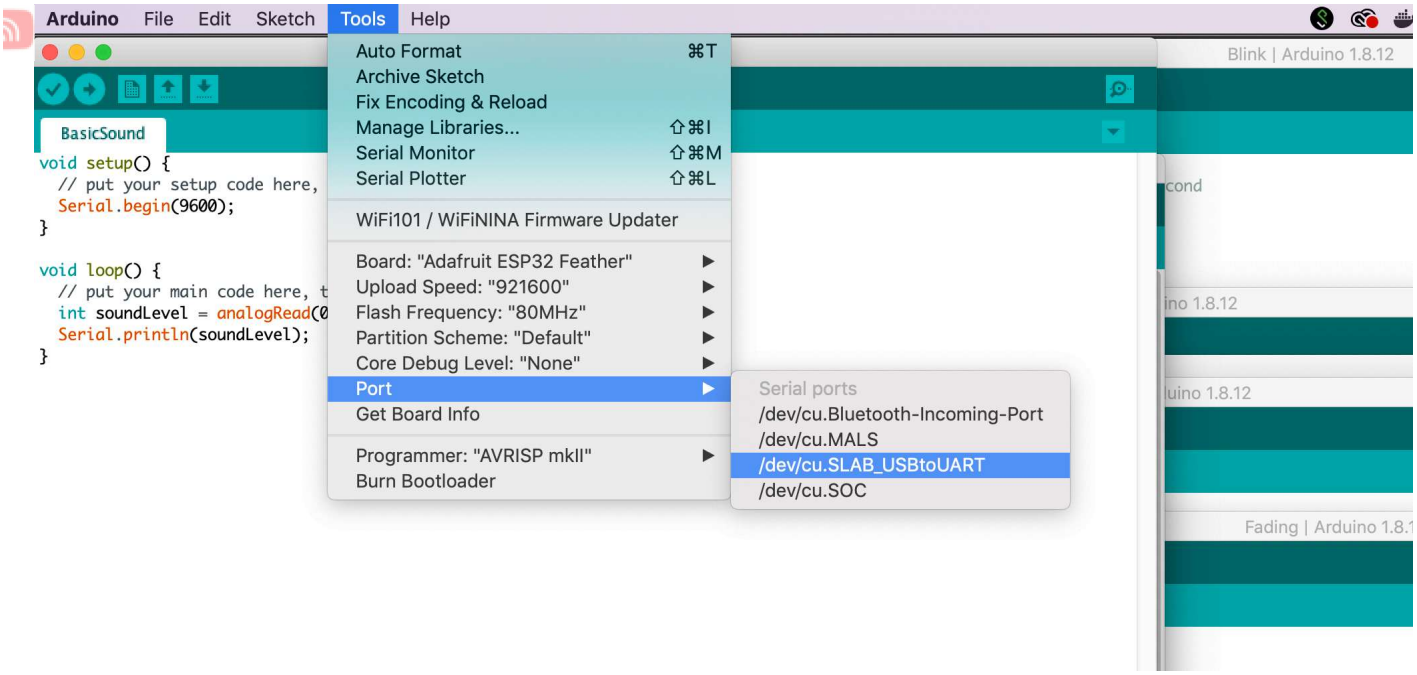
STEP 3: SELECT "ADAFRUIT ESP32 FEATHER" IN BOARD MENU

Once installed, select the `Adafruit ESP32 Feather` in the Board menu.



STEP 4: SELECT THE APPROPRIATE PORT

Finally, select the appropriate port



Resources

Official ESP32 Documentation

- API Reference
- API Guides
- Arduino core for ESP32

Other

- Sparkfun's ESP32 Thing Hookup Guide. Written for Sparkfun's ESP32 Thing board but has relevant WiFi and BLE examples.

Next Lesson

In the next lesson, you will write your first ESP32 program using the ESP32 Arduino library.

Next: Blink an LED with ESP32

MAKEABILITY LAB

DESIGN USE BUILD

MHCI+D human-computer interaction + design

PAUL G. ALLEN SCHOOL

This website was developed by Professor Jon E. Froehlich and the Makeability Lab using Just the Docs. If you found the website useful or use it in your teaching, we'd love to hear from you: jonf@cs.uw.edu. This website and all code is open source (website GitHub, Arduino GitHub, p5js GitHub). You can find the MakeabilityLab_Arduino_Library here. Found an error? File a GitHub Issue.